



Factory Pattern and Singleton Pattern Refactoring

Student Information

Name Surname: Melek İRmak Demirhindi

Student Number: 2481.011025

Department: Management Information Systeme

Submission Date: May 15, 2026

1. Introduction

In this homework I studied Factory Pattern and Singleton Pattern with simple Java food ordering system. In the beginning system was working but code design was not very good. Notification system had many if else statements so code was becoming harder to read and improve.

Main aim of this homework was understanding how design patterns improve software systems. During this homework I learned that design patterns can make projects cleaner and easier to maintain.

In this project first I analyzed problems in original code. Then I refactored system step by step using Factory Pattern and Singleton Pattern. I also used GitHub and created commits during development process.

At first I was confused about interfaces and object creation logic. But after trying different examples and fixing mistakes, I understood the logic better

2. Explanation of the Initial Code Problem

The original code was working but it had many design problems. In the beginning I did not fully understand why the teacher called this code “not well designed”. But after examining the code carefully I understood the problems more clearly.

The biggest problem in the code was the long if else structure.

```
if (notificationType.equalsIgnoreCase("EMAIL")) {
    System.out.println("Sending EMAIL notification...");
    System.out.println("Dear " + studentName +
        ", your order for " + foodName +
        " has been received by " + universityName + ".");
}
else if (notificationType.equalsIgnoreCase("SMS")) {
    System.out.println("Sending SMS notification...");
    System.out.println("Hi " + studentName +
        ", your campus food order is confirmed.");
}
else if (notificationType.equalsIgnoreCase("PUSH")) {
    System.out.println("Sending PUSH notification...");
    System.out.println("New notification: Your " + foodName
        +
        " order is being prepared.");
}
else {
    System.out.println("Unknown notification type.");
}
}
```

At first this may look normal because there are only 3 notification types. But in real software systems applications usually grow bigger with time. For example if the university later wants to add:

- WhatsApp notification
- Telegram notification
- Mobile application notification
- Discord notification
- Voice call notification

then the developer must again open the same class and add more if else blocks. This situation creates many problems.

2.1 Problem of Too Many If Else Statements

Too many if else statements make the code:

- harder to read
- harder to debug
- harder to maintain
- more confusing
- longer
- easier to break accidentally

For example when another developer opens the file, they must read all conditions one by one. In large systems there can be hundreds of lines like this.

Another problem is that if we accidentally type one condition wrong, the system may stop working correctly.

For example:

"SMS"

If somebody writes:

"smss"

then the notification will not work.

2.2 Why One Class Should Not Do Everything

In original code all notification logic was inside CampusFoodOrderService class.

This means one class was responsible for:

- creating order
- printing order information
- deciding notification type
- sending notification

This is not good design because one class doing too many things. If notification system changes, CampusFoodOrderService also needs change. This makes code more dependent and harder to maintain.

2.3 Difficulty of Extending the System

Another problem was extensibility. If we add new notification type, we need edit main class again. This violates Open Closed Principle.

Open Closed Principle means software should be open for extension but closed for modification. But in this system every new notification changes old code. This can create bugs. For example while adding Telegram notification, Email notification can stop working too

2.4 Violated Software Design Principles

The original code weakens or violates several design principles

Single Responsibility Principle

One class should only have one responsibility but CampusFoodOrderService had too many responsibilities.

Open Closed Principle

The class was not closed for modification. Every new notification type required changing old code.

Scalability Problem

If the system grows larger later, this structure becomes difficult to manage.

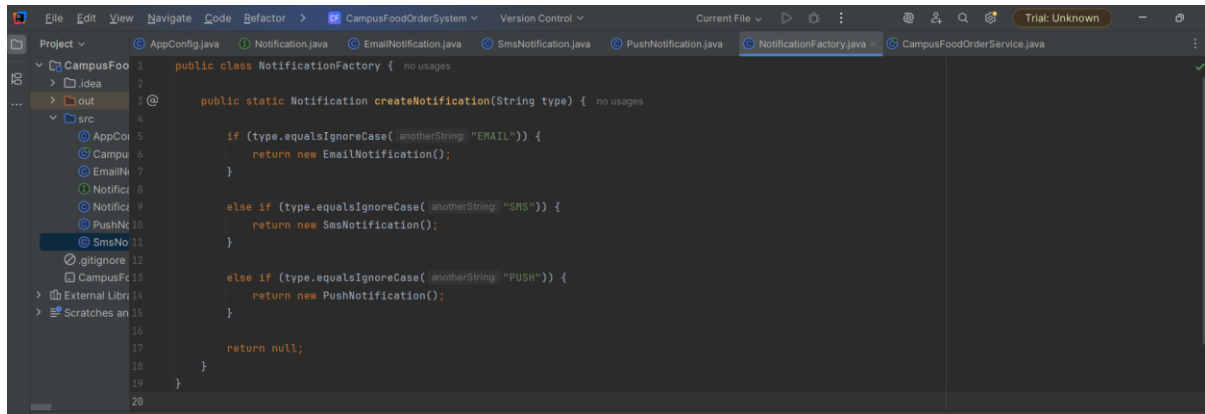
3. Factory Pattern Explanation

Before this homework I heard the name Factory Pattern before but I did not fully understand why people use it. After doing this project I understood that Factory Pattern mainly solves object creation complexity.

3.1 What Problem Does Factory Pattern Solve?

Factory Pattern solves the problem of creating objects directly inside business logic. Without Factory Pattern, the main class creates objects manually.

For example:



```
new EmailNotification()  
new SmsNotification()  
new PushNotification()
```

When the number of objects increases the code becomes messy. Factory Pattern moves this object creation process into a separate class this makes the code cleaner.

3.2 Separating Object Creation From Business Logic

Business logic and object creation should not always stay together.

CampusFoodOrderService should focus on food orders, not creating notification objects.

Factory Pattern separates these jobs. Service class handles orders and factory class creates correct objects. This makes project more organized.

3.3 Real Life Analogy

A real life example for Factory Pattern can be clothing store chain.

For example customer goes to clothing store and says:

“I want black hoodie.”

Customer does not know where product comes from or who prepares it. Store system automatically prepares correct product.

Similarly in Factory Pattern, service class only asks for correct notification object. Then factory creates:

- EmailNotification

- SmsNotification
- PushNotification

This makes code cleaner and easier to improve.

3.4 How Factory Pattern Helps This System

Factory Pattern improved this system very much. Because now:

- notification logic is separated
- code is easier to read
- adding new notification is easier
- main class becomes shorter
- maintenance becomes easier

For example if I want to add WhatsAppNotification later, I only need:

1. Create WhatsAppNotification class
2. Add it inside NotificationFactory

The rest of the system stays same this is much safer.

4. Singleton Pattern Explanation

Singleton Pattern was another important part of this homework. At first I mixed Singleton with static classes but later I understood the difference better. Singleton Pattern ensures that only one object exists in the whole application.

4.1 What Problem Does Singleton Solve?

Singleton solves the problem of creating multiple unnecessary objects. For example university name and delivery fee are shared information. All classes should use same values.

If every class creates its own configuration object:

```
new AppConfig()
```

then:

- memory waste happens
- inconsistent data may happen
- code repetition increases

Singleton prevents this.

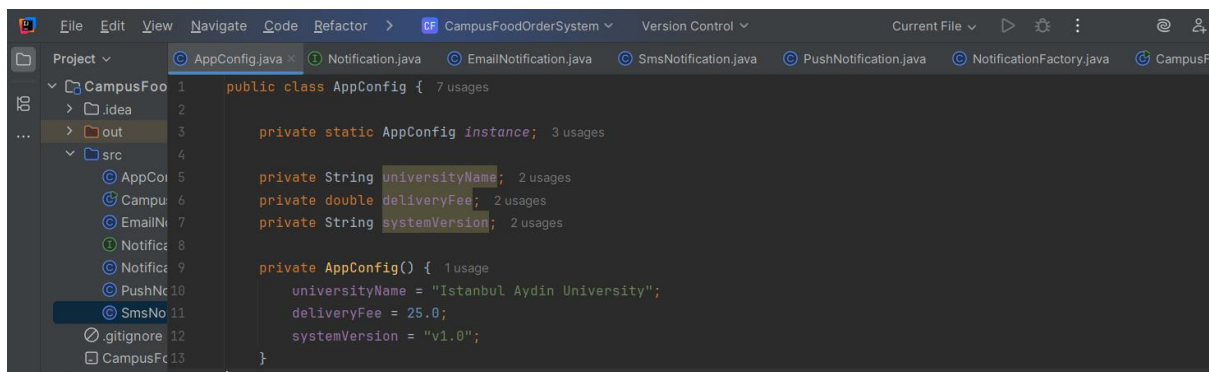
```
AppConfig config = AppConfig.getInstance();
```

4.2 Why Shared Configuration Object Is Useful

In this project AppConfig stores:

- universityName
- deliveryFee
- systemVersion

These values are global for the whole application. If one class stores delivery fee as 25 and another class stores 35, the system becomes inconsistent. Singleton keeps everything centralized. Every class accesses same configuration object.



4.3 Real Life Analogy

A real life analogy for Singleton can be school database system. There is only one central database. Every teacher and student connects to same database. Nobody creates their own separate university database.

Similarly Singleton creates one shared instance.

4.4 Disadvantages of Singleton

Singleton is useful but overusing it can also create problems.

For example:

- global access may increase dependency
- testing becomes harder sometimes
- classes can become tightly coupled
- too many Singleton classes may reduce flexibility

So Singleton should only be used when really necessary.

5. Step By Step Refactoring Process

In this section I will explain how I refactored the project step by step.

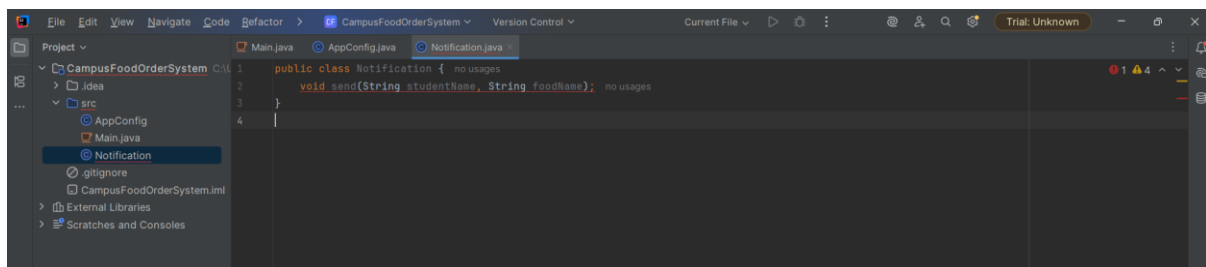
5.1 Creating Notification Interface

First I created a Notification interface.

```
1 public interface Notification { no usages 3 implementations
2
3     void send(String studentName, String foodName); no usages 3 implementations
4 }
```

This interface created a common structure for all notification classes.

At first I created a normal class instead of an interface because I did not fully understand the difference between them. Later I learned that using an interface was better for creating a common structure between different notification types. After understanding the logic better, I changed the class into a Notification interface.



5.2 Creating EmailNotification Class

Then I created EmailNotification class.

```
1 public class EmailNotification implements Notification { no usages
2
3     @Override no usages
4     public void send(String studentName, String foodName) {
5
6         AppConfig config = AppConfig.getInstance();
```

I moved all email related code from the main service class into this class. This made the service class cleaner.

5.3 Creating SmsNotification Class

```
1 public class SmsNotification implements Notification { no usages
2
3     @Override no usages
4     public void send(String studentName, String foodName) {
5
6         System.out.println("Sending SMS notification...");
7
8         System.out.println(
9             "Hi " + studentName +
10            ", your campus food order is confirmed."
11        );
12    }
13 }
```

5.4 Creating PushNotification Class

Then I created PushNotification class.

At first I accidentally wrote:

```
1 public class PushNotification implements Notifications { 1 usage 1 related problem
2
3     @Override no usages
4     public void send(String studentName, String foodName) {
5
6         System.out.println("Sending PUSH notification...");
7     }
```

instead of:

```
1 public class PushNotification implements Notification { no usages
2
3     @Override no usages
4     public void send(String studentName, String foodName) {
5
6         System.out.println("Sending PUSH notification...");
7     }
```

Because of this I got compilation error. After checking the error message I corrected it.

This mistake helped me understand interfaces better.

5.5 Creating NotificationFactory Class

Then I created NotificationFactory class. This was the most important part of Factory Pattern.

```
public class NotificationFactory { no usages  
@ public static Notification createNotification(String type) { no usages
```

I create:

createNotification(String type) method.

This method decides which notification object should be created.

Example:

```
if (type.equalsIgnoreCase( anotherString: "EMAIL")) {  
    return new EmailNotification();  
}
```

This removed object creation responsibility from service class.

5.6 Updating CampusFoodOrderService

Finally I updated CampusFoodOrderService.

Now instead of long if else blocks, the service class only asks factory for notification object.

Example:

```
Notification notification =  
    NotificationFactory.createNotification(notificationType);
```

Then:

```
NotificationFactory.createNotification(notifi  
  
if (notification != null) {  
    notification.send(studentName, foodName);  
}
```

This structure looked much cleaner compared to original version.

6. Singleton Refactoring Process

6.1 Creating AppConfig Class

```
1 public class AppConfig { 7 usages
```

This class stores shared system information.

6.2 Creating Private Constructor

Then I made constructor private.

```
private AppConfig() { 1 usage
```

private constructor prevents creating objects outside the class. So users cannot write:

```
new AppConfig()
```

This is important for Singleton logic.

6.3 Creating Static Instance

Then I created:

```
private static AppConfig instance; 3 usages
```

This stores the single object.

6.4 Creating getInstance() Method

Then I created:

```
public static AppConfig getInstance() { 2 usages  
  
    if (instance == null) {  
        instance = new AppConfig();  
    }  
}
```

This method checks if instance exists.

If not:

```
instance = new AppConfig();
```

Then returns same object.

6.5 Using AppConfig Inside Other Classes

Singleton Pattern solved the problem of repeating shared values like university name and delivery fee in different classes.

7. Final Java Code

Notification Interface

```
1 public interface Notification { no usages 3 implementations
2
3     void send(String studentName, String foodName); no usages 3 implementations
4 }
```

EmailNotification

```
public class EmailNotification implements Notification { no usages
    @Override no usages
    public void send(String studentName, String foodName) {
        AppConfig config = AppConfig.getInstance();

        System.out.println("Sending EMAIL notification...");

        System.out.println(
            "Dear " + studentName +
            ", your order for " + foodName +
            " has been received by " +
            config.getUniversityName() + "."
        );
    }
}
```

SmsNotification

```
public class SmsNotification implements Notification { no usages

    @Override no usages
    public void send(String studentName, String foodName) {

        System.out.println("Sending SMS notification...");

        System.out.println(
            "Hi " + studentName +
            ", your campus food order is confirmed."
        );
    }
}
```

PushNotification

```
public class PushNotification implements Notification { 1 usage

    @Override 1 usage
    public void send(String studentName, String foodName) {

        System.out.println("Sending PUSH notification...");

        System.out.println(
            "New notification: Your " +
            foodName +
            " order is being prepared."
        );
    }
}
```

NotificationFactory

```
public class NotificationFactory { no usages

    public static Notification createNotification(String type) { no usages

        if (type.equalsIgnoreCase( anotherString: "EMAIL")) {
            return new EmailNotification();
        }

        else if (type.equalsIgnoreCase( anotherString: "SMS")) {
            return new SmsNotification();
        }

        else if (type.equalsIgnoreCase( anotherString: "PUSH")) {
            return new PushNotification();
        }

        return null;
    }
}
```

AppConfig

```
1  public class AppConfig { 7 usages
2
3      private static AppConfig instance; 3 usages
4      private String universityName; 2 usages
5      private double deliveryFee; 2 usages
6      private String systemVersion; 2 usages
7
8      private AppConfig() { 1 usage
9          universityName = "Istanbul Aydin University";
10         deliveryFee = 25.0;
11         systemVersion = "v1.0";
12     }
13     public static AppConfig getInstance() { 2 usages
14
15         if (instance == null) {
16             instance = new AppConfig();
17         }
18         return instance;
19     }
20     public String getUniversityName() { 1 usage
21         return universityName;
22     }
23     public double getDeliveryFee() { 1 usage
24         return deliveryFee;
25     }
26
27     public String getSystemVersion() { no usages
28         return systemVersion;
29     }
30 }
31
```

Updated CampusFoodOrderService

```
1 ▶ public class CampusFoodOrderService {
2
3     public void placeOrder(String studentName, 3 usages
4         String foodName,
5         String notificationType) {
6
7         AppConfig config = AppConfig.getInstance();
8
9         System.out.println("Order created for: " + studentName);
10        System.out.println("Food: " + foodName);
11        System.out.println("Delivery fee: "
12            + config.getDeliveryFee() + " TL");
13
14        Notification notification =
15            NotificationFactory.createNotification(notificationType);
16
17        if (notification != null) {
18            notification.send(studentName, foodName);
19        }
20
21        else {
22            System.out.println("Unknown notification type.");
23        }
24
25        System.out.println("-----");
26    }
27
28 ▶ public static void main(String[] args) {
29
30        CampusFoodOrderService service =
31            new CampusFoodOrderService();
32    }
```

```

32
33     service.placeOrder(
34         studentName: "Ali",
35         foodName: "Chicken Sandwich",
36         notificationType: "EMAIL"
37     );
38
39     service.placeOrder(
40         studentName: "Zeynep",
41         foodName: "Vegetarian Pizza",
42         notificationType: "SMS"
43     );
44
45     service.placeOrder(
46         studentName: "Omar",
47         foodName: "Coffee",
48         notificationType: "PUSH"
49     );
50 }
51 }

```

8. Screenshots of Program Output

```

Run CampusFoodOrderService
C:\Users\irmak\.jdk\openjdk-25.0.2\bin\java.exe --javaagent:C:\Users\irmak\AppData\Local\Programs\IntelliJ IDEA 2025.3.3\lib\idea_rt.jar=54435 -Dfile.encoding=UTF-8 -Dsun.stdout
Order created for: Ali
Food: Chicken Sandwich
Delivery fee: 25.0 TL
Sending EMAIL notification...
Dear Ali, your order for Chicken Sandwich has been received by Istanbul Aydin University.
-----
Order created for: Zeynep
Food: Vegetarian Pizza
Delivery fee: 25.0 TL
Sending SMS notification...
Hi Zeynep, your campus food order is confirmed.
-----
Order created for: Omar
Food: Coffee
Delivery fee: 25.0 TL
Sending PUSH notification...
New notification: Your Coffee order is being prepared.
-----
Process finished with exit code 0

```

melekdmrh Remove extraneous character from README.md

Preview Code Blame 64 lines (39 loc) · 1.5 KB

SoftwareDesingAssignment

Code revesion assignment

This project is a simple Java-based campus food ordering system developed for the Software Design Patterns homework.

The main purpose of the project is to demonstrate how Factory Pattern and Singleton Pattern can improve software design and code organization.

In the system, students can place food orders and receive different types of notifications such as:

Email notification SMS notification Push notification

The project was refactored from an initial poorly designed version into a cleaner and more maintainable structure.

Factory Pattern is used to create notification objects.

Instead of creating objects directly inside the main service class, the system uses NotificationFactory to generate the correct notification type.

This makes the code:

cleaner easier to maintain easier to extend

Singleton Pattern is used for the AppConfig class. This class stores shared system information such as:

university name delivery fee system version Only one AppConfig object exists during the application lifecycle.

How to Compile the Project Open terminal inside project folder and run: javac *.java

How to Run the Project After compiling, run: java CampusFoodOrderService

Order created for: Ali Food: Chicken Sandwich Delivery fee: 25.0 TL Sending EMAIL notification...

Order created for: Zeynep Food: Vegetarian Pizza Delivery fee: 25.0 TL Sending SMS notification...

Order created for: Omar Food: Coffee Delivery fee: 25.0 TL Sending PUSH notification...

<> Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insights Settings

SoftwareDesingAssignment Public

Pin Watch 0

main

1 Branch 0 Tags

Go to file

T

Add file

<> Code

melekdmrh Remove extraneous character from README.md

68338a0 · 16 minutes ago

4 Commits

README.md

Remove extraneous character from README.md

16 minutes ago

assignment

Add Campus Food Order Service with notifications

3 hours ago

README

SoftwareDesingAssignment

Code revesion assignment

This project is a simple Java-based campus food ordering system developed for the Software Design Patterns homework.

The main purpose of the project is to demonstrate how Factory Pattern and Singleton Pattern can improve software design and code organization.

In the system, students can place food orders and receive different types of notifications such as:

Email notification SMS notification Push notification

The project was refactored from an initial poorly designed version into a cleaner and more maintainable structure.

Factory Pattern is used to create notification objects.

Instead of creating objects directly inside the main service class, the system uses NotificationFactory to generate the correct notification type.

This makes the code:

10. Personal Reflection

During this homework I understood that working code is not enough. Code also should be clean and easy to maintain.

At first I thought Factory Pattern was not necessary because original code was already working. But after adding different classes I understood why design patterns are important in big software projects.

Most difficult part for me was understanding interfaces and separating responsibilities correctly.

One mistake I made was forgetting parameters inside Notification interface. Because of this some classes gave errors.

Another mistake was writing wrong interface name in PushNotification class.

I fixed these mistakes by reading compiler errors carefully.

I also learned GitHub commit history is important because it shows development process.

I used some online resources and AI tools while researching, but I tried understanding logic myself instead of copying directly.

Final code became:

- cleaner
- shorter
- easier to understand
- easier to extend
- more professional

Compared to original code, refactored version is more organized. Notification logic separated into different classes and shared values centralized with Singleton Pattern.

11. Conclusion

In conclusion this homework helped me understand the importance of software design patterns in real software development.

Before this assignment I only knew design patterns theoretically. But after applying Factory Pattern and Singleton Pattern inside a Java project, I understood how they improve code structure.

Factory Pattern helped separate object creation from business logic and reduced complexity in the main service class.

Singleton Pattern helped create one shared configuration object for the whole system.

The final version of the project became more maintainable, scalable and organized.

I also improved my Java programming, GitHub usage and debugging skills during this homework.

Overall this homework was useful for understanding how professional software systems are designed and managed.

Github links:

[melekdmrh/SoftwareDesingAssignment: Code revesion assignment](#)